



FAST — National University of Computer and Emerging Sciences

Karachi Campus

Department of Artificial Intelligence and Data Science

AL-2002 — Artificial Intelligence Lab

LAB 01

Introduction to AI and Python Programming

Prepared by

Khadeejah Ashraf

Instructor, Department of Artificial Intelligence and Data Science

Fall 2026

Table of Contents

1 Objective.....	3
2 Introduction to Artificial Intelligence.....	3
3 Implementation Platform for AI.....	3
3.1 History of Python.....	4
3.2 Distinct Features of Python.....	4
3.3 IDEs for Python.....	5
3.4 Spyder vs Jupyter Notebook vs Google Colab.....	6
4 Basic Python Syntax.....	6
4.1 Assigning Values to Variables.....	6
4.2 Taking User Input.....	6
4.3 Basic Data Types.....	6
4.4 Conditional Statements.....	9
4.5 Loops.....	9
4.6 Functions.....	10
4.6.1 Types of Functions (Based on Arguments).....	11
4.6.2 Types of Functions (Based on Number of Parameters).....	11
5 Lab Tasks.....	13
5.1 Task 1: Student Profile Builder and Type Inspector.....	13
5.2 Task 2: Semester Result Sheet with Grade Statistics.....	13
5.3 Task 3: Text Analyser for an AI Pre-processing Step.....	14
5.4 Task 4: Number Toolkit Using Functions.....	14
5.5 Task 5: Rule-Based AI Advisor (Capstone).....	15
6 Home Tasks and Grading Rubrics.....	16
6.1 Home Task 1: Debug the Broken Program.....	16
6.2 Home Task 2: Electricity Bill Calculator.....	17
6.3 Home Task 3: Lab Inventory Manager.....	17
6.4 Home Task 4: Pattern and Series Generator.....	18
6.5 Home Task 5: Rule-Based AI Ticket Router.....	19

1 Objective

1. Introduce Artificial Intelligence and expose students to different AI problems and their solutions.
2. Acquire programming skills in Python and get introduced to the Python libraries used for Artificial Intelligence.
3. Get familiar with the different IDEs available for Python programming.
4. Solve some basic AI problems using the Python programming language.

2 Introduction to Artificial Intelligence

The term Artificial Intelligence was first coined decades ago, in 1956, by John McCarthy at the Dartmouth Conference. He defined AI as “the science and engineering of making intelligent machines.” In other words, Artificial Intelligence is the science of getting machines to think and make decisions like humans. In the recent past, AI has been able to accomplish this by creating machines and robots that are used in a wide range of fields including healthcare, robotics, marketing, business analytics and many more.

3 Implementation Platform for AI

Artificial Intelligence has been around for over half a century and its advancements are growing at an exponential rate. The demand for AI is at its peak, and this lab course will help you understand the concepts of AI through practical implementation in Python. Python has made its way into the most complex technologies such as Artificial Intelligence, Machine Learning and Deep Learning.

“Python is an experiment in how much freedom programmers need. Too much freedom and nobody can read another's code; too little and expressiveness is endangered.” — Guido van Rossum

Python is the top programming language in both the TIOBE and PYPL indices. According to PYPL, which publishes a separate ranking for five countries, Python is the leading language in all five (United States, India, Germany, United Kingdom and France).

Jan 2023	Jan 2022	Programming Language	Ratings	Change
1	1	Python	16.36%	+2.78%
2	2	C	16.26%	+3.82%
3	4	C++	12.91%	+4.62%
4	3	Java	12.21%	+1.55%
5	5	C#	5.73%	+0.05%
6	6	Visual Basic	4.64%	-0.10%
7	7	JavaScript	2.87%	+0.78%
8	9	SQL	2.50%	+0.70%
9	8	Assembly language	1.60%	-0.25%

Jan 2023	Jan 2022	Programming Language	Ratings	Change
10	11	PHP	1.39%	-0.00%

Figure 1: TIOBE Index — top ten programming languages (January 2023).

3.1 History of Python

- Invented in the Netherlands in the early 1990s by Guido van Rossum.
- Named after the comedy series Monty Python.
- Open sourced from the very beginning.
- Considered a scripting language, but is much more than that.
- Scalable, object oriented and functional from the beginning.
- Used by Google from its early days.
- Increasingly popular across research and industry.

3.2 Distinct Features of Python

Python is a dynamic, high level, free, open source and interpreted programming language. It supports object-oriented as well as procedural programming. In Python it is not necessary to declare the type of a variable, because it is a dynamically typed language.

- **Less code:** implementing AI involves a large number of algorithms. Thanks to Python's support for pre-defined packages, we do not have to code these algorithms from scratch. Python also encourages a “check as you code” methodology that reduces the burden of testing.
- **Prebuilt libraries:** Python has hundreds of pre-built libraries for Machine Learning and Deep Learning. To run an algorithm on a dataset, you only need to install and load the necessary package with a single command. Examples include NumPy, Keras, TensorFlow and PyTorch.
- **Ease of learning:** Python uses a very simple syntax that can express both simple computations, such as the concatenation of two strings, and complex processes such as building a Machine Learning model.
- **Platform independent:** Python runs on multiple platforms including Windows, macOS, Linux and Unix. When transferring code from one platform to another, packages such as PyInstaller take care of dependency issues.
- **Massive community support:** Python has a huge community of users, which is always helpful when we encounter coding errors, along with many groups and forums where programmers help each other.

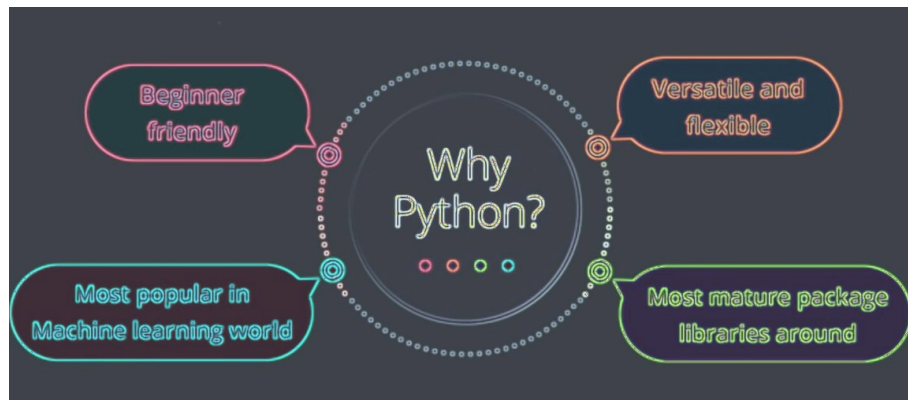


Figure 2: Why Python is the preferred language for AI.

3.3 IDEs for Python

An Integrated Development Environment (IDE) is a software suite that consolidates the basic tools required to write, run and test software. The most widely used IDEs and editors for Python are shown below.

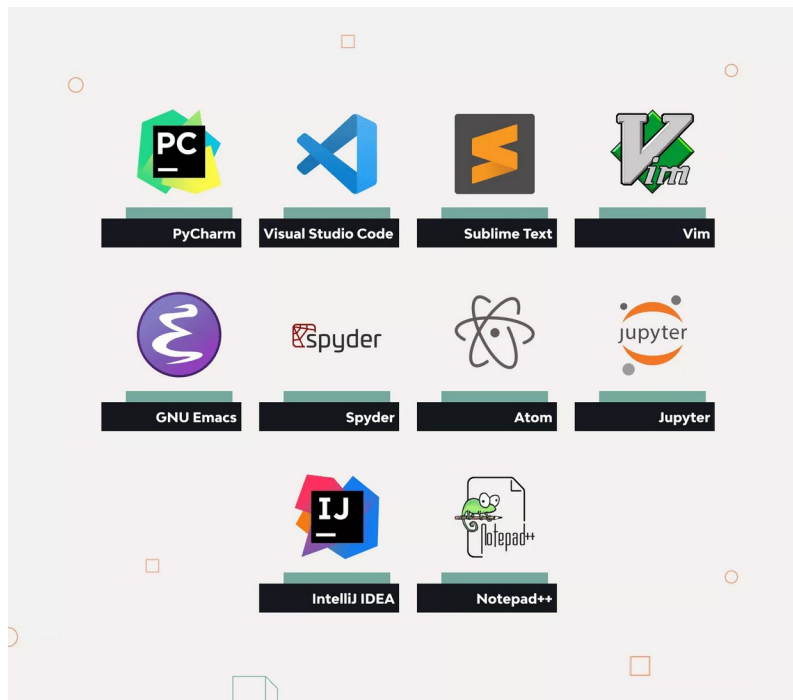


Figure 3: Popular IDEs and editors for Python development.

3.4 Spyder vs Jupyter Notebook vs Google Colab

The three environments used most often in this lab differ mainly in how they are launched and where the computation takes place.

#	Feature	Spyder	Jupyter Notebook	Google Colab
1	Installation	Yes	Yes	No
2	Launch	GUI	Web browser	Web browser
3	Internet connection	No	No	Yes
4	Usage	Development	Data Science	Data Science
5	Visualisation	NA	Yes	Yes
6	Data	Local / Remote	Local / Remote	Google Drive
7	Power	CPU	CPU	CPU / GPU / TPU

Table 1: Comparison of Spyder, Jupyter Notebook and Google Colab.

4 Basic Python Syntax

The traditional starting point in any language is printing a message to the screen. In Python this needs a single statement.

```
print("Hello, World!")
```

4.1 Assigning Values to Variables

A variable is created the moment a value is assigned to it using the assignment operator (=). The type of the variable is inferred automatically from the value.

```
a = 2          # 'a' is a variable; the integer value 2 is assigned using (=)
b = 3.3        # 'b' is a variable; the float value 3.3 is assigned using (=)
c = 'khadeejah' # 'c' is a variable; the string value 'khadeejah' is assigned
                using (=)
```

4.2 Taking User Input

The input() function reads a value entered by the user from the keyboard and always returns it as a string.

```
name = input("Enter your name: ")
print(name)
```

4.3 Basic Data Types

Numeric Data Types

```
int_data = 17
float_data = 12.6
complex_data = 14 + 5j

print('Data : ', int_data, ' ', Type : ' ', type(int_data))
print('Data : ', float_data, ' ', Type : ' ', type(float_data))
print('Data : ', complex_data, ' ', Type : ' ', type(complex_data))
```

Text Data Types

```
string_data = 'Machine Learning'
string_data_alphabet = 'a'
string_paragraph = 'Machine learning is a subfield of artificial intelligence.'
string_num = '12'

print('Data : ', string_data, ' ', Type : ' ', type(string_data))
print('Data : ', string_data_alphabet, ' ', Type : ' ', type(string_data_alphabet))
print('Data : ', string_paragraph, ' ', Type : ' ', type(string_paragraph))
print('Data : ', string_num, ' ', Type : ' ', type(string_num))
```

Fetching a Specific Character or the Complete String

```
t = 'Hello World!'

print(t)           # Prints the complete string
print(t[0])        # Prints the first character of the string
print(t[2:5])      # Prints characters from the 3rd to the 5th
print(t[2:])       # Prints the string starting from the 3rd character
print(t * 2)       # Prints the string two times
print(t + "TEST")  # Prints the concatenated string
```

Data Types Used to Store Collections

- List
- Tuple
- Set
- Range
- Dictionary

```
list_data = [11, 12, 13, 14, 15, 'orange']
tuple_data = (12, 15, 17, 'khadeejah')
set_data = {'khadeejah', 'python', 'ai', 12, 18.6}
range_data = range(1, 11)

print('List Data : ', list_data)
print('Tuple Data : ', tuple_data)
print('Set Data : ', set_data)
print('Range Data : ', range_data)
```

List

```
list1 = ['physics', 'chemistry', 1997, 2000] # a list can store any type of data
list2 = [1, 2, 3, 4, 5, 6, 7]

print("list1[0] : ", list1[0]) # Fetching the item at index 0
print("list2[1:5] : ", list2[1:5]) # Fetching the items from index 1 to 4

# Updating an item
my_list = ['physics', 'chemistry', 1997, 2000]
print("Value available at index 2 : ", my_list[2])
my_list[2] = 2001
print("New value available at index 2 : ", my_list[2])

# Deleting an item
my_list = ['physics', 'chemistry', 1997, 2000]
print(my_list)
del my_list[2]
print("After deleting the value at index 2 : ", my_list)
```

Tuple

```
my_tuple = ('abcd', 786, 2.23, 'khadeejah', 70.2)
tinytuple = (123, 'khadeejah')

print(my_tuple) # Prints the complete tuple
print(my_tuple[0]) # Prints the first element of the tuple
print(my_tuple[1:3]) # Prints the elements from the 2nd till the 3rd
print(my_tuple[2:]) # Prints the elements starting from the 3rd element
print(tinytuple * 2) # Prints the tuple two times
print(my_tuple + tinytuple) # Prints the concatenated tuple

# A list can be updated, but a tuple cannot
my_list = ['abcd', 786, 2.23, 'khadeejah', 70.2]
my_list[2] = 1000 # Valid syntax with a list
my_tuple[2] = 1000 # Invalid: raises TypeError with a tuple
```

Dictionary


```
my_dict = {}
my_dict['coll'] = "This is col one"
my_dict[2] = "This is col two"
print('=== Created Dictionary === ', my_dict)

tinydict = {'name': 'Khadeejah', 'age': 23, 'dept': 'AI & DS'}

print(my_dict['coll'])      # Prints the value for the 'coll' key
print(my_dict[2])          # Prints the value for the key 2
print(tinydict)            # Prints the complete dictionary
print(tinydict.keys())     # Prints all the keys
print(tinydict.values())   # Prints all the values
```

4.4 Conditional Statements

if Statement

```
age = 18

if age >= 18:
    print("You are eligible to vote.")
```

if-else Statement

```
age = 18

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

Multiple Conditions (if-elif-else)

```
amount = int(input("Enter amount: "))

if amount < 1000:
    discount = amount * 0.05
    print("Discount", discount)
elif amount < 5000:
    discount = amount * 0.10
    print("Discount", discount)
else:
    discount = amount * 0.15
    print("Discount", discount)

print("Net payable:", amount - discount)
```

4.5 Loops

for Loop

```
for var in list(range(5)):
    print(var)

# Other examples
for letter in 'Python':      # traversal of a string sequence
    print('Current Letter :', letter)

fruits = ['banana', 'apple', 'mango']
for fruit in fruits:         # traversal of a list sequence
    print('Current fruit :', fruit)

print("Good bye!")
```

while Loop

```
count = 0

while count < 9:
    print('The count is:', count)
    count = count + 1

print('Good Bye')
```

4.6 Functions

A function is a block of reusable code that performs a single, related action. Functions are defined with the `def` keyword and are executed only when they are called.

Non-Parametric Function

```
# Function definition
def non_para_func():
    print("This is a non-parametric function")

non_para_func()
```

Parametric Function

```
# Function definition
def para_func(a, b):
    """This function takes the values of a and b provided by the caller."""
    return a + b

para_func(1, 2)
```

Return Statement

```
# Function definition
def add_numbers(arg1, arg2):
    # Add both the parameters and return the result
    total = arg1 + arg2
    print("Inside the function : ", total)
    return total

# Now the function can be called
total = add_numbers(10, 20)
print("Outside the function : ", total)

# A function that returns nothing
def print_message(message):
    print('Print Statement : ', message)
    return # This is equivalent to 'return None'

result = print_message("Hello, World!")
print(result) # Output: None
```

4.6.1 Types of Functions (Based on Arguments)

- Positional arguments
- Default arguments
- Keyword arguments

```
# Positional arguments – values are mapped strictly by position
def details(name, age):
    print('Name : ', name)
    print('Age : ', age)

details(23, 'Khadeejah') # name = 23 and age = 'Khadeejah' – the order is wrong!

# Default argument
def details(name, age=23):
    print('Name : ', name)
    print('Age : ', age)

details('Khadeejah') # age takes its default value 23

# Keyword arguments – order no longer matters
def details(name, age, height):
    print('Name : ', name)
    print('Age : ', age)
    print('Height : ', height)

details(name='Khadeejah', age=23, height=5.5)
```

4.6.2 Types of Functions (Based on Number of Parameters)

- Fixed length parameters
- Variable length parameters

Fixed Length Parameters

```
# Function with fixed parameters
def add(a, b):
    '''a and b are fixed parameters'''
    return a + b

add(1, 2, 3)  # Providing extra arguments raises a TypeError
```

Variable Length Parameters

```
# Variable length arguments (*args)
def details(*numbers):
    print('Numbers : ', numbers)

details(1, 2, 3)

# Keyword variable length arguments (**kwargs)
def details(**name):
    print('Names : ', name)

details(n1='Khadeejah', n2='Maryam', n3='Hira')
```

5 Lab Tasks

The following five tasks must be completed during the lab session. They are based only on the concepts covered in this manual: variables, user input, data types, collections, string handling, conditional statements, loops and functions. Every task requires you to write and execute your own Python code — copying an example from the manual without extending it will not earn credit. Save each task in a separate file named Lab01_Task<number>.py (or in a separate notebook cell) and show the running program to your instructor.

5.1 Task 1: Student Profile Builder and Type Inspector

Problem statement: Write a program that collects a student's details from the keyboard and stores them in a single dictionary, then prints a formatted profile card along with the data type of every stored value. Use Khadeejah, age 23, as your test student.

Requirements and constraints:

- Read four values with input(): name, age, CGPA and enrolled courses (courses are entered as one comma-free space-separated line, e.g. AI DS OOP).
- Convert the age to int and the CGPA to float; the courses must be stored as a list and the whole profile as a dictionary.
- Print each key together with its value and its type using type().
- Derive and print the student's initials using string indexing only (no library calls).
- Print the profile twice using string repetition to create a separator line, for example a row of dashes.
- Do not import any module.

Expected outcome: A dictionary such as {'name': 'Khadeejah', 'age': 23, 'cgpa': 3.75, 'courses': ['AI', 'DS', 'OOP']} printed as a neat profile card, followed by a type report showing str, int, float and list respectively.

5.2 Task 2: Semester Result Sheet with Grade Statistics

Problem statement: A semester result is stored as a list of tuples, where each tuple holds a subject name and the marks obtained out of 100. Write a program that validates the marks, converts them into grades and produces a summary of how many subjects fall in each grade.

Requirements and constraints:

- Start from a list of at least six (subject, marks) tuples that you define yourself.
- Use a loop with if–elif–else to map marks to grades: 85 and above A, 70–84 B, 60–69 C, 50–59 D, below 50 F.
- Any marks outside the range 0–100 must be treated as invalid: skip that subject, count it separately and report it at the end.
- Maintain a dictionary whose keys are grades and whose values are the number of subjects in that grade; update it inside the loop.
- Compute the average of the valid marks without using any imported function.
- Print a subject-wise table and then the grade summary and the average, rounded by using integer arithmetic or plain division.

Expected outcome: A subject-by-subject listing with grades, a dictionary such as {'A': 2, 'B': 3, 'C': 1}, the number of invalid entries and the class average printed at the end.

5.3 Task 3: Text Analyser for an AI Pre-processing Step

Problem statement: Before text can be used by an AI model it must be analysed and cleaned. Write a program that takes a sentence from the user and reports several statistics about it using loops, strings and collections.

Requirements and constraints:

- Read one sentence using `input()`. Use “Khadeejah is learning Artificial Intelligence” as your test sentence.
- Count the vowels and the consonants using a single loop over the characters.
- Build a set containing the unique characters of the sentence (spaces excluded) and print how many unique characters there are.
- Build a dictionary that maps each character to the number of times it occurs, and print only the characters that occur more than once.
- Reverse the sentence using a while loop and string concatenation — do not use any built-in reversing function.
- Print the first five and the last five characters of the sentence using slicing.

Expected outcome: A report showing the vowel and consonant counts, the set of unique characters, the frequency dictionary of repeated characters, the reversed sentence and the two slices.

5.4 Task 4: Number Toolkit Using Functions

Problem statement: Build a small toolkit of functions that analyse numbers. This task tests all three kinds of arguments and both fixed and variable length parameter lists.

Requirements and constraints:

- `is_prime(n)` — returns True or False using a loop; it must not import anything.
- `digit_sum(n)` — returns the sum of the digits of n using a while loop and integer division.
- `classify(n, threshold=100)` — uses a default argument and returns the string 'small' or 'large' depending on the threshold.
- `summarise(*numbers)` — accepts any count of numbers and returns a dictionary containing the smallest value, the largest value and the average, all computed with your own loop rather than a built-in.
- `describe(**details)` — accepts keyword arguments and prints each key with its value; call it at least once as `describe(name='Khadeejah', age=23)`.
- Call `classify()` once positionally, once with the default argument and once using a keyword argument, and print the three results.
- Every function must return a value or explicitly return None; state in a comment which functions return None.

Expected outcome: Console output showing the primes found in a range you choose, the digit sums, the three different calls to `classify()`, the dictionary produced by `summarise()` and the keyword report from `describe()`.

5.5 Task 5: Rule-Based AI Advisor (Capstone)

Problem statement: Rule-based agents were the earliest form of artificial intelligence: they map an input pattern to an action using a fixed knowledge base. Build a text-based advisor that answers a student's questions about the AI Lab by matching keywords against a knowledge base and that reports statistics about the session when it ends.

Requirements and constraints:

- Store the knowledge base as a dictionary in which each key is a keyword (for example 'python', 'grading', 'project') and each value is the answer the agent should give.
- Run the agent inside a while loop that keeps asking for a question until the user types exit.
- Use the membership operator `in` and a for loop over the knowledge base to decide which rule fires; if no rule fires, print a default fallback answer.
- Record every question in a list and count the fired and unmatched rules in separate variables.
- Implement the matching logic as a function `match_rule(question, knowledge_base)` that returns the answer or `None`, and a function `session_report(*questions, **stats)` that prints the summary.
- Greet the user by name at the start using `input()`; use Khadeejah, age 23, when demonstrating the program.

Expected outcome: An interactive session in which at least three questions are answered from the knowledge base and one falls through to the default answer, followed by a session report listing the questions asked, the number matched and the number unmatched.

6 Home Tasks and Grading Rubrics

The following five home tasks are to be completed individually and submitted before the next lab. The five tasks together carry 10 marks: each task is worth 2 marks and is graded against the four-criterion rubric printed beneath it, with each criterion worth 0.5 marks and awarded in full or not at all. Submit one .py file per task, named Lab01_HomeTask<number>_<yourRollNumber>.py, together with a screenshot of the output. Code that does not run will be graded only on the criteria it can satisfy.

Home Task	Title	Marks
1	Debug the Broken Program	2
2	Electricity Bill Calculator	2
3	Lab Inventory Manager	2
4	Pattern and Series Generator	2
5	Rule-Based AI Ticket Router	2
	Total	10

Table 2: Marks distribution for the home tasks (10 marks in total).

6.1 Home Task 1: Debug the Broken Program

Problem statement: The program below is meant to store a student record and print it, but it fails to run. Copy it, find every error, correct it and add a short comment above each correction explaining what was wrong and why your fix works.

```
student = ('Khadeejah', 23, 3.75)
student[1] = 24                                # the student had a birthday

def show(name, age):
    print('Name : ', name)
    print('Age : ', age)

show(23, 'Khadeejah')

age_entered = input('Enter age: ')
if age_entered > 18:
    print('Adult')

marks = [90, 85, 78]
print('Average : ', total / 3)
```

Requirements and constraints:

- Identify at least four distinct errors and classify each one as a syntax error, a type error, a logical error or a name error.
- Keep the tuple as a tuple: solve the update problem without converting the record into a list, and explain in a comment why the original line failed.

- The corrected program must run from top to bottom and print the record, the age check and the correct average.

Expected outcome: A working program plus a comment block listing every error found, its category and the reasoning behind the fix.

Criterion	Marks (out of 2)
At least four distinct errors identified and correctly classified	0.5
Corrected program runs without errors and prints all three required outputs	0.5
Tuple immutability and the input() type conversion handled correctly	0.5
Accurate, specific explanatory comments and readable, indented code	0.5
Total	2

6.2 Home Task 2: Electricity Bill Calculator

Problem statement: Write a program that calculates a monthly electricity bill from the number of units consumed, using a slab-based tariff and a set of reusable functions.

Requirements and constraints:

- Tariff: first 100 units at 12 per unit, the next 200 units at 18 per unit, and everything above 300 units at 25 per unit. Only the units inside each slab are charged at that slab's rate.
- Write `compute_units_cost(units)` that returns the slab cost, and `compute_bill(units, tax_rate=0.17, fixed_charge=150)` that uses default arguments and returns the final payable amount.
- Call `compute_bill()` three times: once with only the units, once overriding the tax rate with a keyword argument, and once overriding both defaults.
- Read the units with `input()` and reject negative or non-numeric-looking values using a conditional statement before any calculation is done.
- Print an itemised bill: units, slab cost, tax, fixed charge and total.

Expected outcome: Three itemised bills printed for the same consumption under different tax and fixed-charge assumptions, with correct slab arithmetic (for example, 350 units gives a slab cost of 1200 + 3600 + 1250).

Criterion	Marks (out of 2)
Slab arithmetic correct below 100, between 100 and 300, and above 300	0.5
Both functions defined as specified and called with default and keyword arguments	0.5
Input validation rejects negative or invalid input before any calculation	0.5
Itemised output complete and clearly labelled; no repeated blocks in the code	0.5
Total	2

6.3 Home Task 3: Lab Inventory Manager

Problem statement: The AI Lab keeps a record of its equipment. Write a menu-driven program that manages this inventory using the collection types covered in this manual.

Requirements and constraints:

- Store the inventory as a dictionary mapping an item name to a tuple of (quantity, condition), for example {'GPU workstation': (4, 'good')}.
- Offer a menu inside a while loop with the options: add an item, update a quantity, delete an item, search for an item, list everything, and exit.
- Because tuples cannot be modified in place, updating a quantity must rebuild the tuple; add a comment explaining this.
- Maintain a set of all conditions that appear in the inventory and print it when the user lists everything.
- Every menu operation must be implemented as its own function; at least one function must use a default argument.
- Handle the case of an item that does not exist without crashing the program.

Expected outcome: A session in which each of the six menu options is demonstrated at least once, ending with a final listing of the inventory and the set of conditions.

Criterion	Marks (out of 2)
All six menu operations work correctly and the loop exits cleanly	0.5
Correct use of dictionary, tuple and set, including the tuple rebuild on update	0.5
Each operation is a separate function, with at least one default argument	0.5
Missing items and invalid choices handled without a crash; menu clearly displayed	0.5
Total	2

6.4 Home Task 4: Pattern and Series Generator

Problem statement: Write a program that generates numeric patterns and series on demand. This task focuses on nested loops and on variable length parameter lists.

Requirements and constraints:

- triangle(n) prints a right-angled triangle of numbers with n rows using nested for loops.
- multiplication_table(*numbers) prints the table from 1 to 10 for every number passed to it, however many are given.
- fibonacci(limit) uses a while loop to print all Fibonacci numbers below limit and returns them as a list.
- collatz(n) repeatedly halves an even number and applies $3n + 1$ to an odd one until it reaches 1, returning the number of steps taken.
- A single driver section at the bottom must call all four functions; no function may call input() itself.
- No imported modules and no built-in shortcut for generating the series.

Expected outcome: A triangle of the requested height, tables for at least three numbers passed in one call, the Fibonacci list below the chosen limit, and the Collatz step count (for example, $n = 27$ takes 111 steps).

Criterion	Marks (out of 2)
All four functions produce correct output for the values tested	0.5
Correct use of nested loops and of *args in multiplication_table()	0.5
Functions return values where required rather than only printing	0.5
Driver section calls every function; output labelled and free of duplicated logic	0.5
Total	2

6.5 Home Task 5: Rule-Based AI Ticket Router

Problem statement: A university help desk receives text complaints and must route each one to the correct department. Build a rule-based classifier — the simplest form of an AI agent — that reads complaints, assigns a department using keyword rules, and reports how well it performed.

Requirements and constraints:

- Store the rules as a dictionary that maps a department to a list or tuple of trigger keywords, for example 'IT Support' for 'wifi', 'laptop' and 'password'.
- Provide a list of at least eight sample complaints inside the program together with the department each one truly belongs to, so that accuracy can be measured.
- route(complaint, rules, fallback='General Office') must return the department; the fallback must be a default argument.
- Loop over all complaints, compare the predicted department with the true one, and count correct and incorrect predictions.
- Print the accuracy as a percentage and a dictionary showing how many complaints were routed to each department.
- evaluate(*results, **info) must accept the individual outcomes and keyword details such as analyst='Khadeejah' and print the final report.

Expected outcome: A routing table showing each complaint, its predicted department and whether the prediction was correct, followed by the accuracy percentage, the per-department counts and the keyword report from evaluate().

Criterion	Marks (out of 2)
Routing gives the correct department for the eight or more sample complaints	0.5
Rules stored in a dictionary of keyword collections and matched with a loop	0.5
Accuracy percentage and per-department counts computed and printed correctly	0.5
route() and evaluate() use default, *args and **kwargs as specified; clear report	0.5
Total	2